

分组密码算法设计

清华大学计算机系

于红波

2026年3月11日



提纲

- 计算安全
- Feistel结构和SPN结构
- DES算法
- 多重加密-Triple-DES
- SM4
- AES



密码体制的安全性

- 无条件安全性(unconditional security)即完善保密性(perfect security)
 - 即使攻击者有无限的计算资源也不可能攻破密码体制，则该密码体制是无条件安全的
 - OTP是无条件安全的
- 计算安全性(computational security)
 - 破译一个密码体制所做的计算上的努力
 - 如果使用最好的算法破译一个密码体制至少需要 N 次操作（ N 是一个特定的非常大的数字），定义该密码体制是计算安全的
- 可证明安全性(provable security)
 - 通过规约的方式为安全性提供证据
 - 如果可以破译密码体制 A ，则就可以解决一个数学难题 B （分解因子问题，离散对数问题）



规约证明方法

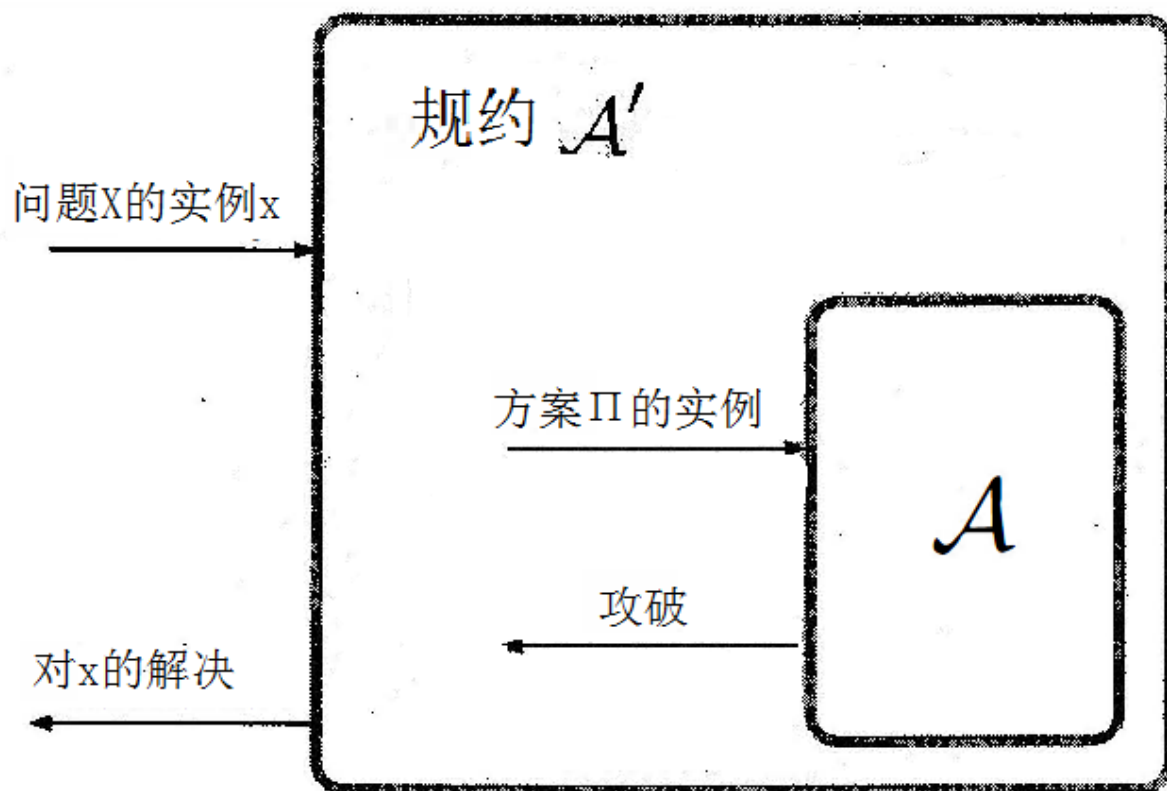
□ 规约证明过程

- 指定一个有效的敌手A的攻击 Π ，将A的成功率 $\varepsilon(n)$ 表示出来
- 构造一个有效算法A'，A'将敌手A作为子程序使用，试图解决难度X
 - A'不需要知道A如何工作
 - A被A'调用时，A所见的视图和A本身和 Π 交互的视图相同
 - 如果A攻破A'模拟的 Π 的实例，那么A'给出解决难题X的实例
- 如果 $\varepsilon(n)$ 不可忽略，那么A'解决X的概率不可忽略，矛盾
- 不存在有效的敌手A可以攻破 Π ，也就是 Π 计算安全



规约证明方法

□ 规约证明过程





现有的计算资源

- ❑ Intel Core-i5, i7 CPUs
 - ❑ Widely used CPUs in 2021 (notebook, desktop computers)
 - ❑ Normally each costs RMB 4000-10000
 - ❑ Each has two or four cores, runs at the speed about 2.5GHz
 - ❑ Each core performs about 2^{32} 64-bit integer (or floating point) operations per second
- ❑ 超算：美国，欧洲国家，中国，日本
 - El Capitan：位于美国劳伦斯利弗莫尔国家实验室，运算速度达到1742 petaflops（每秒174.2亿亿次浮点运算）。



计算安全

- 一种密码术如果不是**数学上**无法破译的，则必须在**实践上**无法破译
 - 实践上无法破译：使用一种不能在“合理的时间”内，以任何“合理的成功率”攻破的方案
 - 如：一个方案理论上能够被攻破，但在200年内，使用最快的可用的超级计算机，不能以大于 2^{-30} 的概率攻破
- 计算安全的概念包括了两种放宽了的完美安全概率
 - 仅仅在对抗“有效”的敌手时，安全性存在
 - 敌手潜在的成功率是非常低的



计算安全

- 假设一个加密方案，密钥空间 K 比消息 M 小得多，无论该方案如何构造，都会存在两种攻击
 - 给定密文 c , 敌手遍历所有的密钥 k , 能够破译 c (给出 c 对应着所有的可能的消息，泄露消息的一些信息)
 - 已知明文的攻击。敌手掌握了密文 $c_1, c_2, \dots, c_t$, 对应着明文 $m_1, m_2, \dots, m_t$. 敌手猜测一个密钥 k , 看是否对所有的 i , 都有 $DEK(c_i) = m_i$.
- 短密钥加密多个消息，要实现安全性
 - 限定敌手的运行时间
 - 不认为非常小的成功概率是一种攻破
- 任何加密方案中密钥空间必须足够大，以至于敌手不能够遍历



密码分析类型

□ 根据敌手获取信息的能力

□ 唯密文攻击(ciphertext-only attack)

- 敌手只能获取一定数量的密文

□ 已知明文攻击(known-plaintext attack)

- 敌手可以获取一定数量的用相同密钥加密的明密文对

□ 选择明文攻击(chosen-plaintext attack)

- 敌手可以选择明文，并得到对应的密文

□ 选择密文攻击(chosen-ciphertext attack)

- 敌手不仅可以选择明文，还可以选择密文，并得到对应的明文



实际的对称密码算法

□ 实际的对称加密算法需要满足两个条件

□ 计算安全

- 攻击者知道所使用的密码体制（Kerckhoff假设）
- 能够抵抗已知明文攻击

□ 容易使用

- 相对短的密钥能够用来加密很多消息

□ 实际的对称加密体制

□ 分组密码（Block Cipher）

□ 序列密码（Stream Cipher）



分组密码

- 代换密码 (substitution cipher)
 - 代换表太小， 26个字母
 - 不能够抵抗唯密文攻击和已知明文攻击
- 如果把代换表增加到 2^{128} 个元素？
 - The resulting cipher is strong!
 - 密钥太长， 没法存储
 - Internation Data Corporation (IDT) 估计：到2009年，全世界产生的数据是 1.2 millision Petabyte (2^{70} byte= 10^{21})
 - 全世界在2018年创建、捕获、复制和消耗的数据总量为33 泽字节 (ZB)，相当于33万亿GB。2020年，这一数字增长到59 ZB，预计到2025年将达到令人难以想象的175 ZB。1ZB是8,000,000,000,000,000,000,000比特。



分组密码算法

- 分组密码可以看成是一个巨大的“代换密码”
 - DES: 2^{64} 个元素的代换表
 - AES: 2^{128} 个元素的代换表
- 表中每个元素只被计算一次
 - 无需存储巨大的表
 - 如何计算每个元素?
 - 等价于: 如何加密一个消息分组?
如何设计一个分组密码算法?



分组密码研究进展

□ DES时代

- 1973年：NBS(NIST)征集加密标准，IBM提交
- 1974年：NBS第二次征集
 - IBM提交64比特密钥的算法
 - NSA加入设计，建议缩减密钥长度
- 1976年：DES被NBS采纳为数据加密标准(FIPS 46)
- 1983年：DES被再次确认为标准
- 1988年：1993年：第二次，第三次确认标准
- 1998年：DES被破译（55小时，暴力攻击）
- 1999年：DES第四次被确认（FIPS 46-3， Triple-DES）



分组密码研究进展（续）

□ AES时代

- 1997年，NIST公开征数据加密算法以取代DES
- 1998年，共收到15个算法
- 1999年，从15个中选中5个算法：MARS、RC6、Rijndael、Serpent和Twofish
- 2000年10月：Rijndael获胜，Vincent Rijmen 和Joan Daemen
- 2001年11月：NIST公布新的数据加密标准AES





分组密码研究进展（续）

- ❑ 欧洲NESSIE工程
 - ❑ 2000年启动，共征集到17个分组密码算法
 - ❑ 2001年，进入第二轮的7个分组密码算法
 - ❑ 2003年，MISTY1、Camellia和SHACAL-2获胜
- ❑ 韩国标准：SEED和ARIA
- ❑ 我国：SMS4（SM4），用于无线局域网产品
- ❑ ISO/IET标准：2006年
 - ❑ 64位：TDEA、MISTY1和CAST-128
 - ❑ 128位：AES、Camellia、SEED
- ❑ 轻量级分组密码算法：
 - ❑ Present、CLEFIA、SIMON、SPECK



分组密码算法

□ n -比特的明文分组加密成 n -比特的密文分组

□ 分组长度 n 比特

□ DES: 64比特

□ AES: 128比特

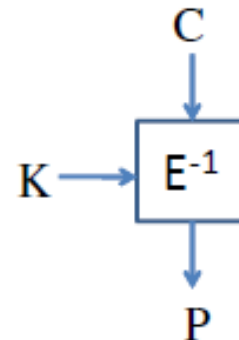
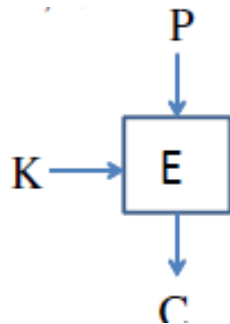
□ 密钥长度

□ DES: 56比特（不安全）

□ AES: 128、192和256比特

加密:

$$C = E_K(P)$$



解密:

$$P = E_K^{-1}(C)$$



分组密码设计原则

□ 安全性原则（Shannon提出）：混淆与扩散--分组密码设计的本质

- 混淆（Confusion）：使得密文和密钥之间的统计关系复杂化，以阻止攻击者发现密钥
- 扩散（Diffusion）：每个密文比特受到所有明文比特的影响，使得明文统计特性消散在密文中

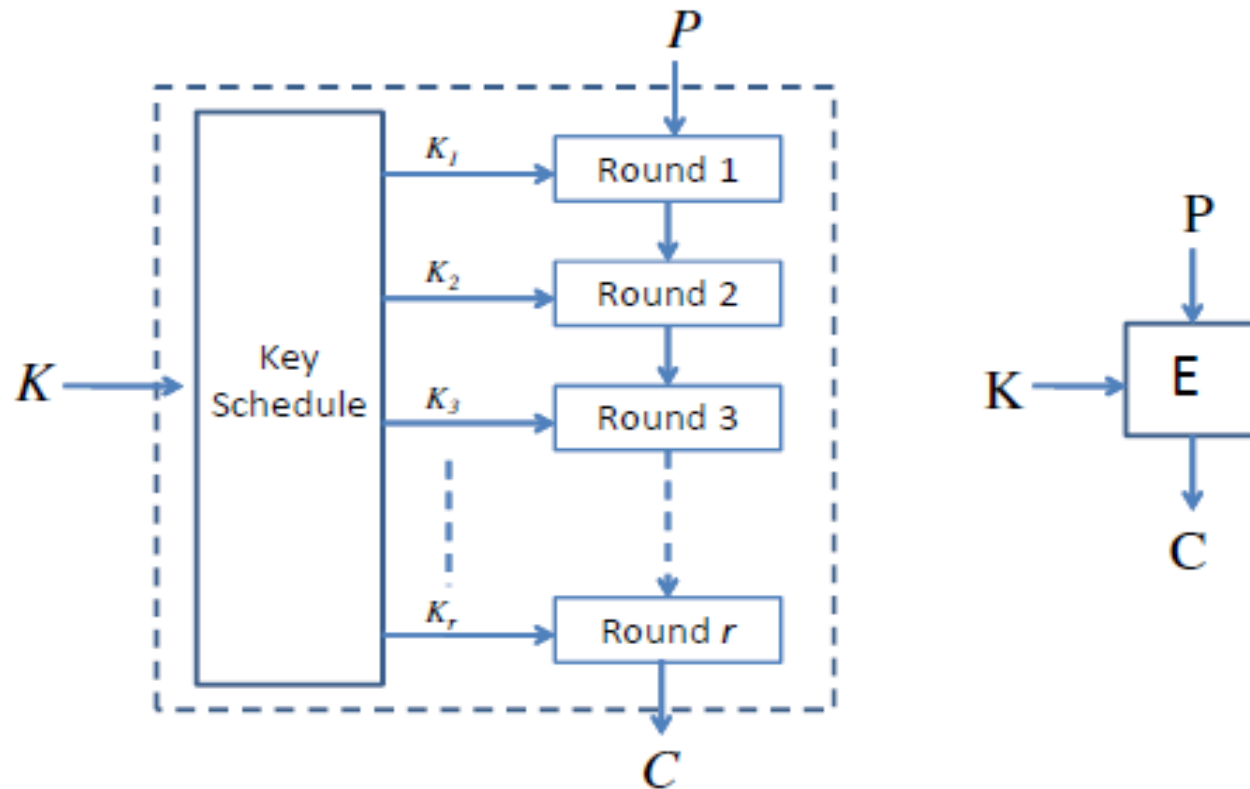
□ 实现混淆扩散

- 一种有效方法——“乘积密码”
- 两种或两种以上基本密码的迭代应用, 例如：交替使用代换(substitution)与置换(permutation)



迭代结构

- 基于轮函数的迭代结构分组密码算法（乘积密码）：便于设计、安全性评估和实现





轮函数

□ 轮函数的设计策略

□ 混淆(Confusion): 非线性部件

- 小的代换表 S-box
- 乘法与异或
- 加与异或
-

□ 扩散(Diffusion): 让所有的比特互相影响

- 线性变换
- 置换
- 移位、循环移位
-



轮函数（续）

□设计轮函数的两种方案

□Feistel Network

□DES

□Substitution-Permutation Network(SPN)

□AES

□其他方案



密钥方案

- 密钥生成方案 (Key Schedule)
 - 多种方法
 - 目前为止没有完美的方法
 - 基本准则：密钥的每个比特应该影响不同位置的很多轮的轮子密钥



Feistel Network

- Feistel Network
 - 由Horst Feistel发明(1973年)
 - DES的设计者之一
- 该结构被广泛使用，AES最终的5个候选算法中有3个使用了Feistel网络（Mars、RC6、Twofish）
- AES (rijndael) 使用SPN结构



Horst Feistel
1915-1990



Feistel Network

□ 加密过程

$$(L_0, R_0) = P$$

$$L_1 = R_0$$

$$R_1 = L_0 \oplus F(K_0, R_0)$$

...

$$L_i = R_{i-1}$$

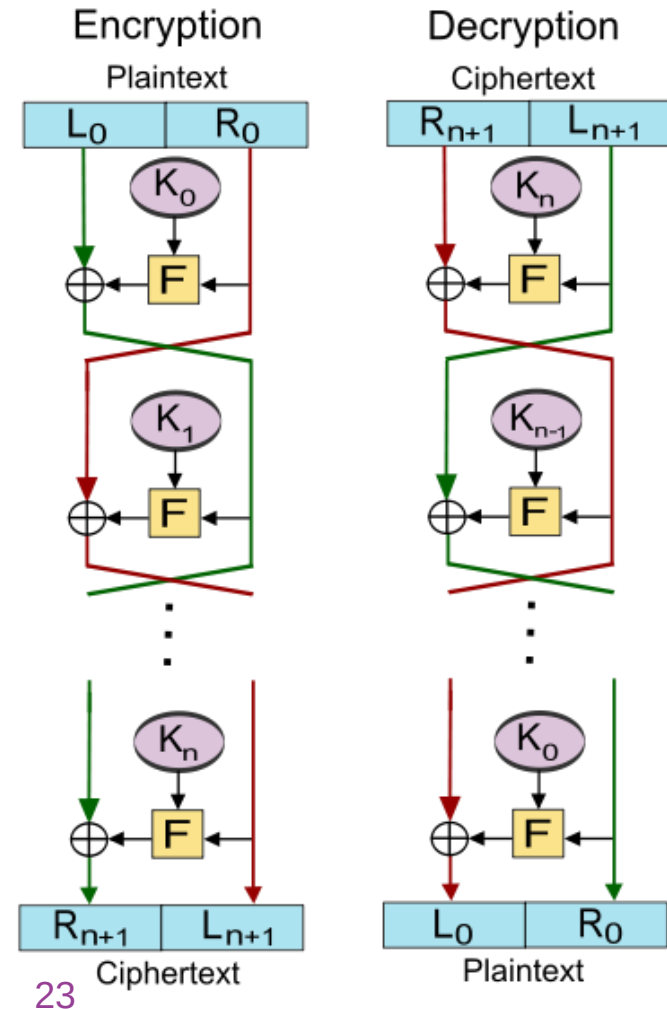
$$R_i = L_{i-1} \oplus F(K_{i-1}, R_{i-1})$$

...

$$L_r = R_{r-1}$$

$$R_r = L_{r-1} \oplus F(K_{r-1}, R_{r-1})$$

$$C = (R_r, L_r)$$





DES: Feistel Network

加解密过程

Encryption:

$$\underline{(L_0, R_0) = P}$$

$$L_1 = R_0$$

$$R_1 = F(\underline{K_1}, L_0)$$

...

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(\underline{K_i}, R_{i-1})$$

...

$$L_r = R_{r-1}$$

$$R_r = L_{r-1} \oplus F(\underline{K_r}, R_{r-1})$$

$$\underline{C = (R_r, L_r)}$$

Decryption:

$$\underline{(L_0, R_0) = C}$$

$$L_1 = R_0$$

$$R_1 = F(\underline{K_r}, L_0)$$

...

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(\underline{K_{r-i+1}}, R_{i-1})$$

...

$$L_r = R_{r-1}$$

$$R_r = L_{r-1} \oplus F(\underline{K_1}, R_{r-1})$$

$$\underline{P = (R_r, L_r)}$$



Feistel Network

□ Feistel结构的属性

□ 可逆性

$$\begin{array}{l} L_i = R_{i-1} \\ R_i = L_{i-1} \oplus F(K_i, R_{i-1}) \end{array} \longleftrightarrow \begin{array}{l} R_{i-1} = L_i \\ L_{i-1} = R_i \oplus F(K_i, L_i) \end{array}$$

□ 加解密使用通一套逻辑：除了密钥使用顺序不用

□ 加密： $K_1, K_2, K_3, \dots, K_r$

□ 解密： $K_r, K_{r-1}, K_{r-2}, \dots, K_1$

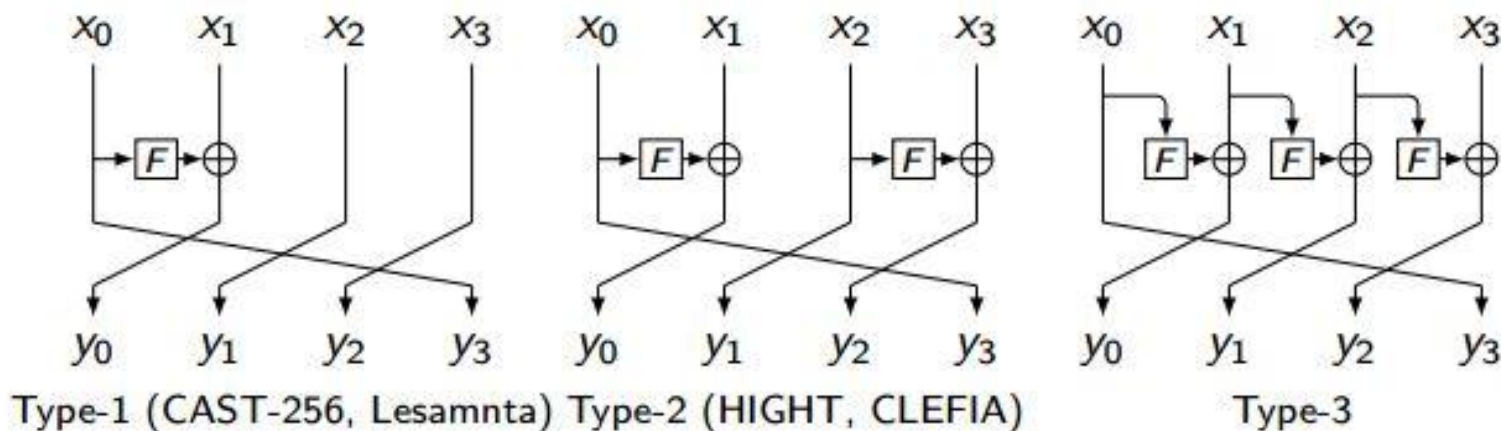


其他Feistel结构

Unbalance Feistel Cipher

L_0 和 R_0 长度不相等：如Skipjack

Generalized Feistel networks: 多分枝





Feistel结构的安全性

□ Michael Luby and Charles Rackoff证明

- 如果轮函数是安全的伪随机函数，将轮密钥 S_i 作为种子，则由Feistel结构构造的分组密码三轮就是一个伪随机置换，4轮是一个强的伪随机置换（即使允许敌手访问逆置换）

□ 使用Feistel结构的算法

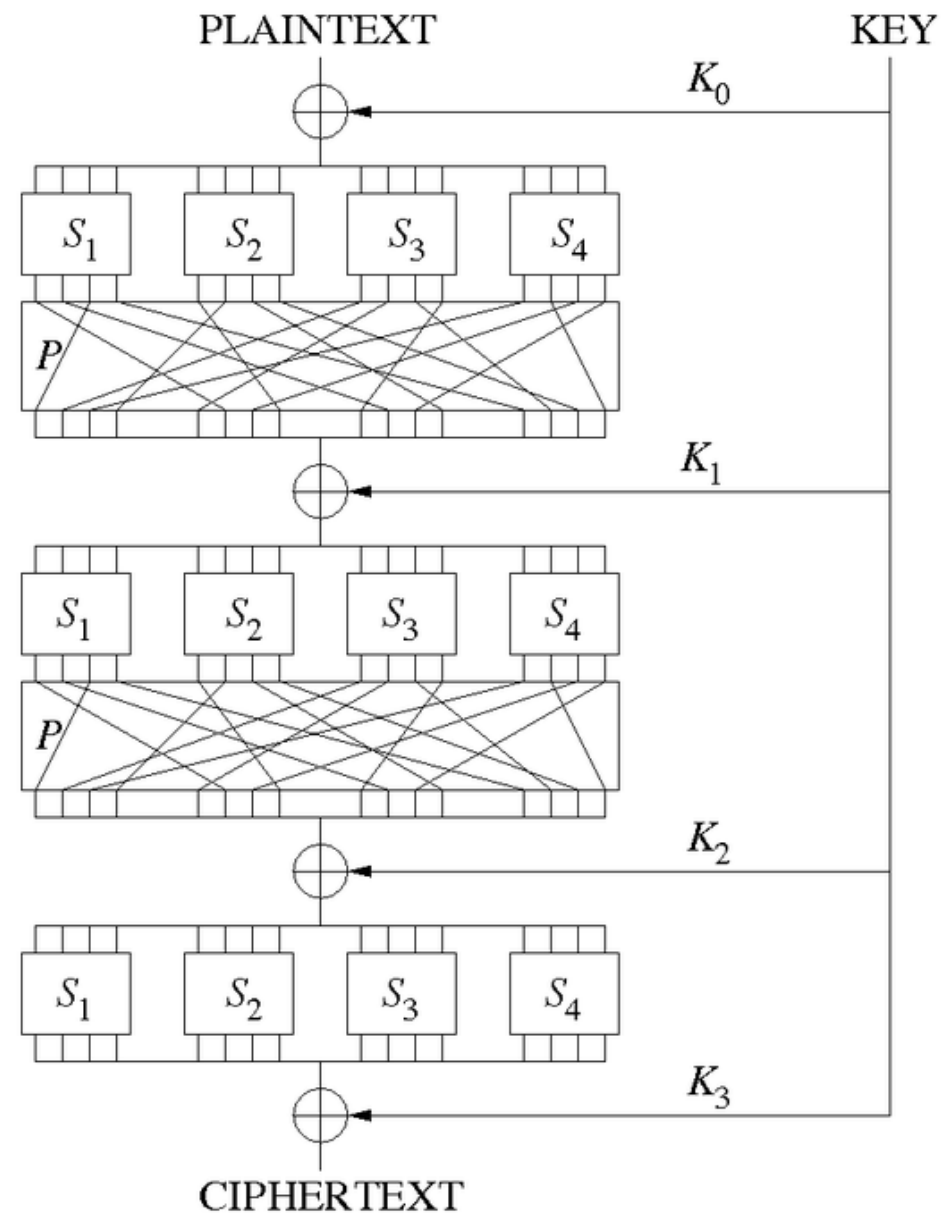
- Feistel or modified Feistel: Blowfish, Camellia, CAST-128, DES, FEAL, GOST 28147-89, ICE, KASUMI, LOKI97, Lucifer, MARS, MAGENTA, MISTY1, RC5, Simon, TEA, Triple DES, Twofish, XTEA
- Generalized Feistel: CAST-256, CLEFIA, MacGuffin, RC2, RC6, Skipjack, SMS4



Substitution–permutation network

SP-network, SPN

- AES, Kuznyechik, PRESENT, SAFER, SHARK, Square等
- S: substitute cipher (confusion)
- P: transposition cipher (diffusion)





Substitution-Permutation Network

□ S盒属性：

- 改变输入的1比特，输出比特中有一半的比特改变
- 输出的每一个比特依赖输入的所有比特

□ P盒属性：

- 好的P置换是使得S盒的输出比特分布到下一轮的尽可能多的S盒
- 扩散：扩散就是让明文中的每一位影响密文中的许多位，或者说让密文中的每一位受明文中的许多位的影响。这样可以隐蔽明文的统计特性。
- 混淆：是指让密文和密钥之间的统计关系变得复杂,使得敌手不能通过密文的统计关系,推测出密钥的统计关系（非线性替换）



DES

- 分组加密算法
- 64比特消息分组
- 56比特密钥
 - 另加 8 个冗余比特用于奇偶检验
- Feistel 结构
- 共16轮 + 起始置换 + 末置换

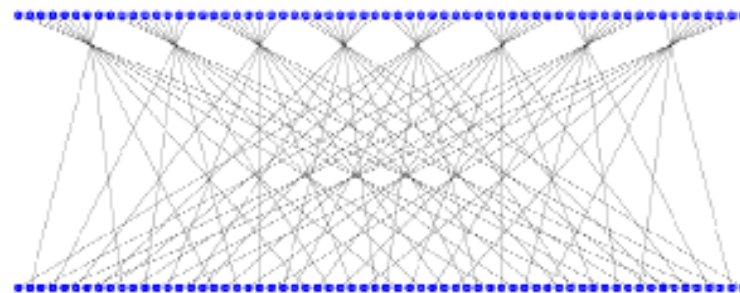
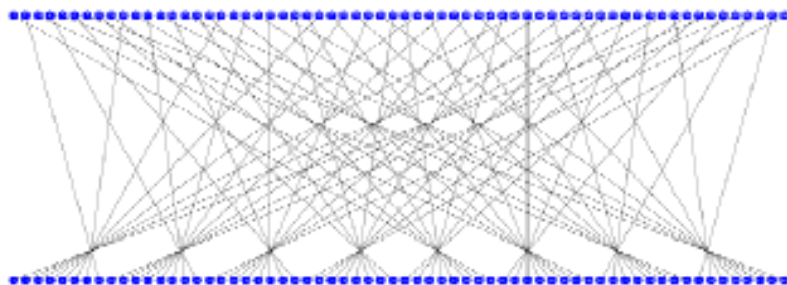




起始置换和末置换

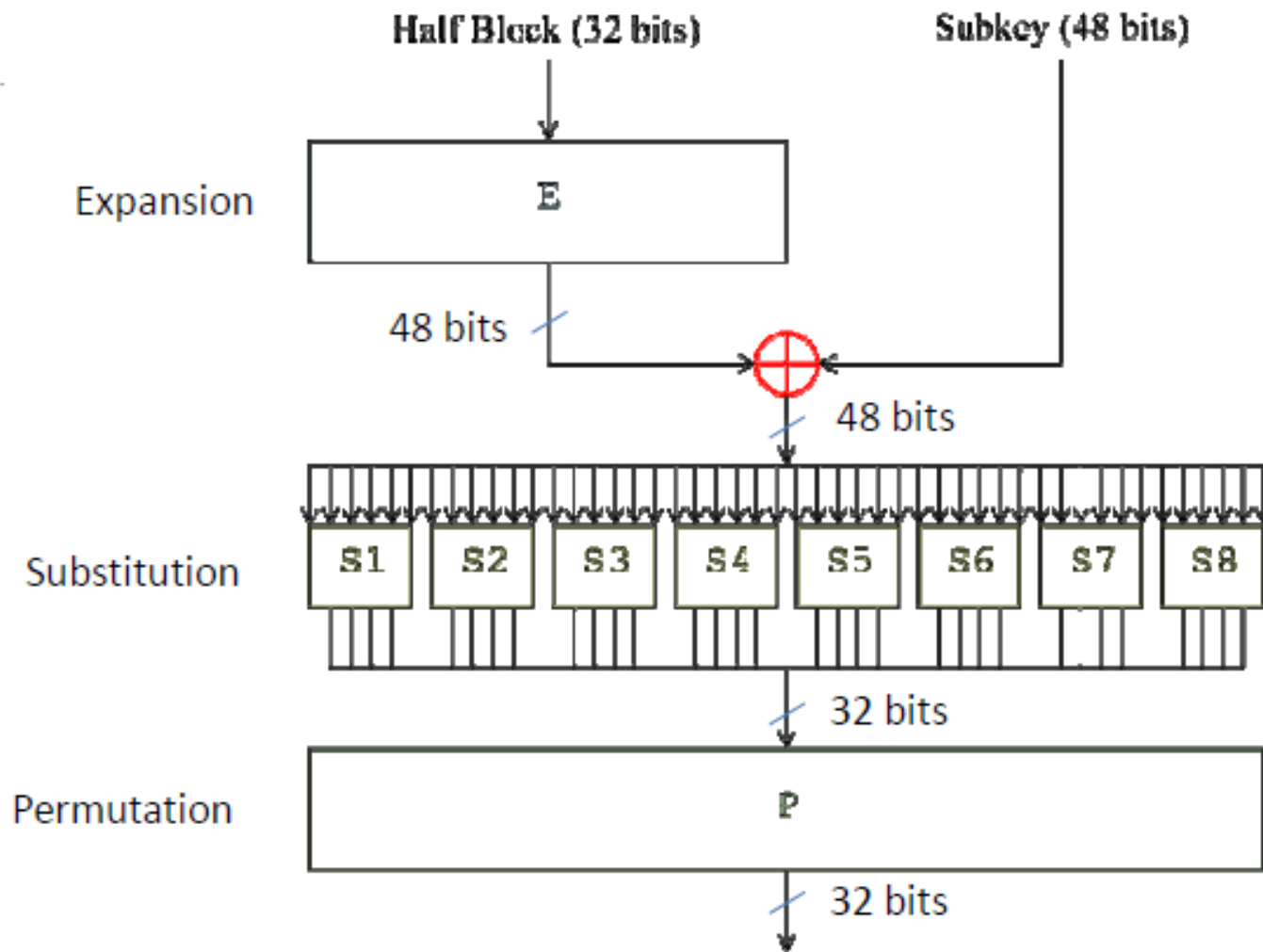
IP							
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

IP^{-1}							
40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25





DES: F 函数

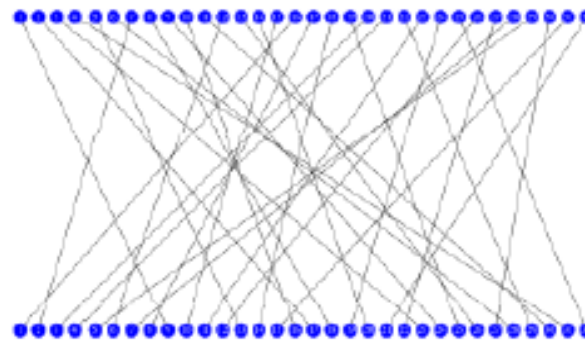
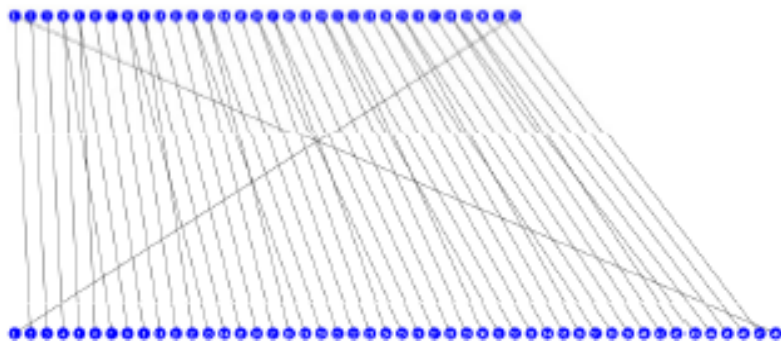




DES: F函数中的扩展和置换

E					
<u>32</u>	<u>1</u>	2	3	<u>4</u>	<u>5</u>
<u>4</u>	<u>5</u>	6	7	<u>8</u>	<u>9</u>
<u>8</u>	<u>9</u>	10	11	<u>12</u>	<u>13</u>
<u>12</u>	<u>13</u>	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	<u>32</u>	<u>1</u>

P			
16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25





DES: S-box

- 使用8个 S 盒： $S_1 S_2 \dots S_8$
每一个S盒 $S_i : \{0,1\}^6 \rightarrow \{0,1\}^4$
用4乘16的矩阵描述
- 6比特的串 $B = b_1 b_2 b_3 b_4 b_5 b_6$
 - $b_1 b_6$ 决定矩阵的行， $b_2 b_3 b_4 b_5$ 决定矩阵的列
 - $C_j = S_j(B_j)$



S1盒

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

S2盒

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9



S3盒

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12

S4盒

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14



S5盒

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3

S6盒

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13



S7盒

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12

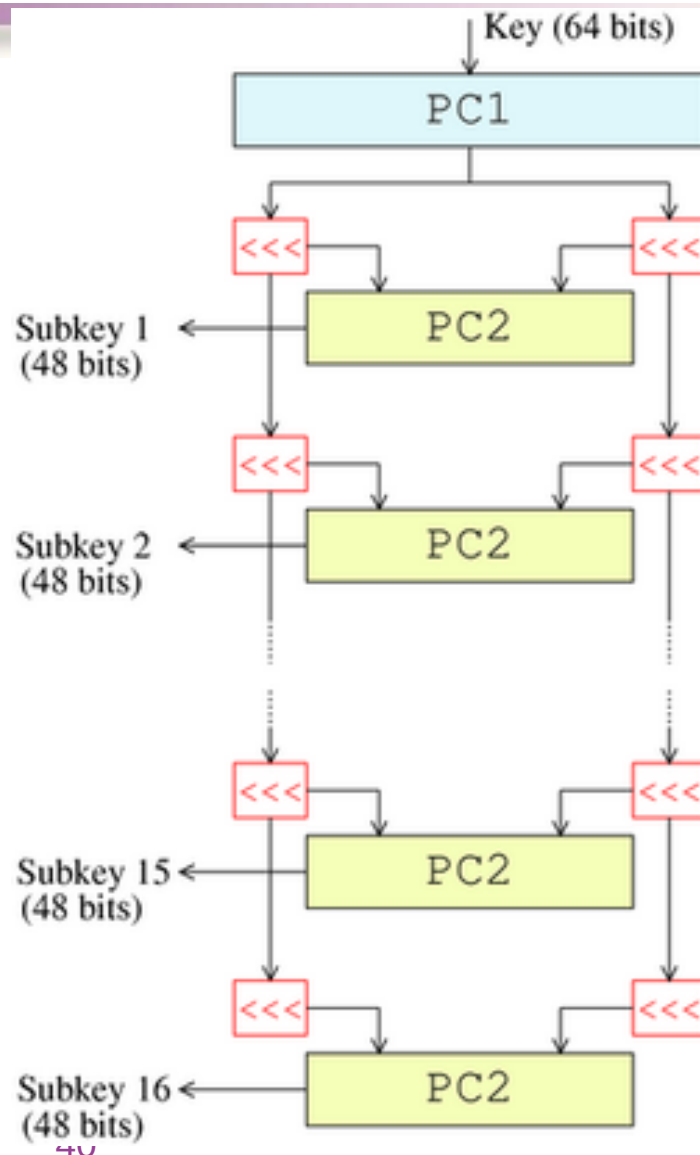
S8盒

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11



DES密钥生成算法

- 当 $i=1,2,9,16$ 时，移位数位1
- 当 $i=3,4,5,6,7,8,10,11,12,13,14,15$ 时，移位数为2
- PC1为一56比特的置换
- PC2从56比特中选择48比特





PC1置换表

57	49	41	33	25	17	9	1	58	50	42	34	26	18
10	2	59	51	43	35	27	19	11	3	60	52	44	36
63	55	47	39	31	23	15	7	62	54	46	38	30	22
14	6	61	53	45	37	29	21	13	5	28	20	12	4

PC2置换表

14	17	11	24	1	5	3	28	15	6	21	10
23	19	12	4	26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40	51	45	33	48
44	49	39	56	34	53	46	42	50	36	29	32



DES 密钥的互补特性

□ 若 $c = Enc_k(m)$, 则 $\bar{c} = Enc_{\bar{k}}(\bar{m})$

□ 选择 $m_1 = \bar{m}_2$, $c_1 = Enc_k(m_1)$, $c_2 = Enc_k(m_2)$,
则 $\bar{c}_2 = Enc_{\bar{k}}(m_1)$

遍历密钥 k :

- 若 $c_1 = Enc_k(m_1)$, 则 k 可能是正确的密钥;
若 $\bar{c}_2 = Enc_k(m_1)$, 则 $\bar{k}$ 可能是正确的密钥;
- 若 $Enc_k(m_1) \neq c_1$ 和 $\bar{c}_2$, 则 k 和 $\bar{k}$ 都不是正确的密钥

- 选择明文攻击
- 一次计算排除两个错误密钥
- 仅需遍历 2^{55} 种密钥的可能



多重加密

- DES: 56比特密钥长度
 - 在今天的计算能力下, 不再安全
 - 如何增加密钥长度?
- 多重加密
 - Double-DES
 - Triple-DES



多重加密

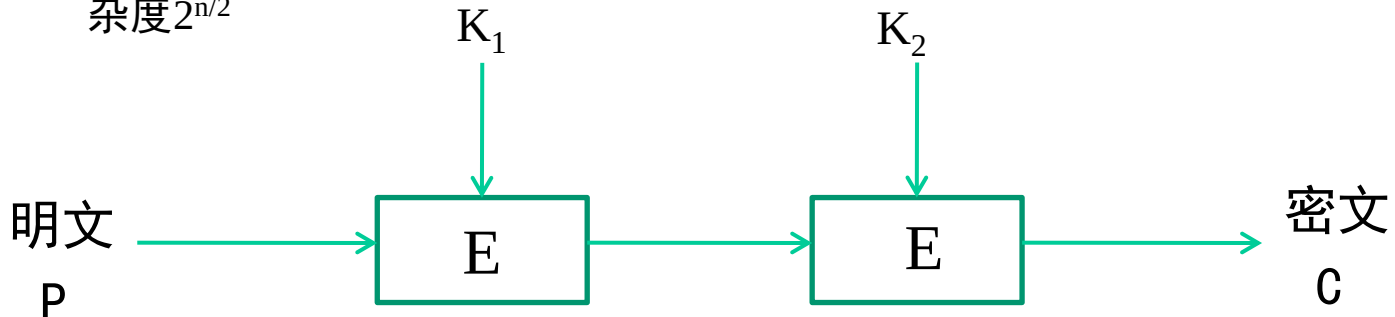
□ Double-DES

- 密钥长度112比特

- 加密: $C = E_{k2}(E_{k1}(P))$

- 不能够抵抗中间相遇攻击: 时间 2^{57} , 存储 2^{56}

- 构造两个集合 $I = \{E_{k1}(P)\}$, $J = \{D_{k2}(C)\}$, 利用生日攻击寻找碰撞, 时间、空间复杂度 $2^{n/2}$





多重加密

□ Triple-DES

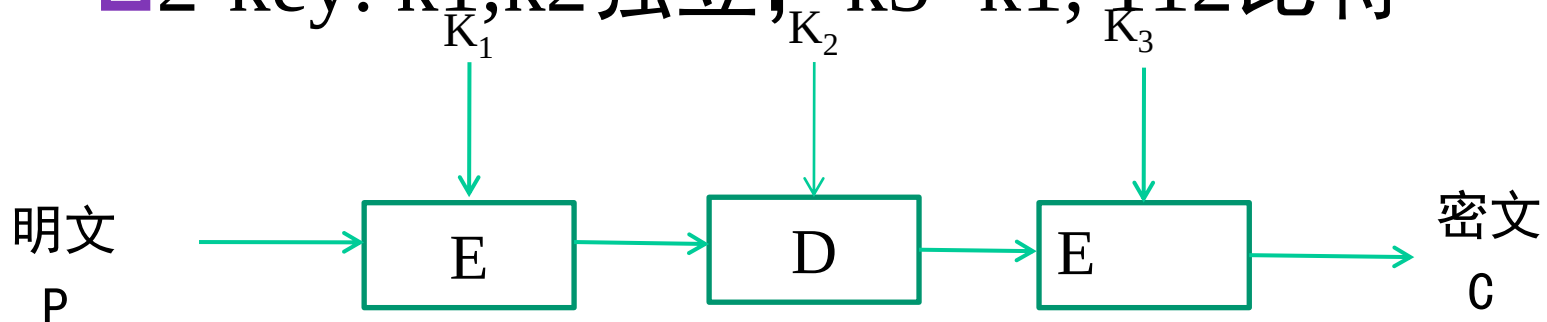
□ 加密: $C = E_{k3}(D_{k2}(E_{k1}(P)))$

□ 解密: $P = D_{k1}(E_{k2}(D_{k3}(C)))$

□ 密钥选择

□ 3-key: $k1, k2, k3$ 互相独立, 168比特

□ 2-key: $k1, k2$ 独立, $k3 = k1$, 112比特



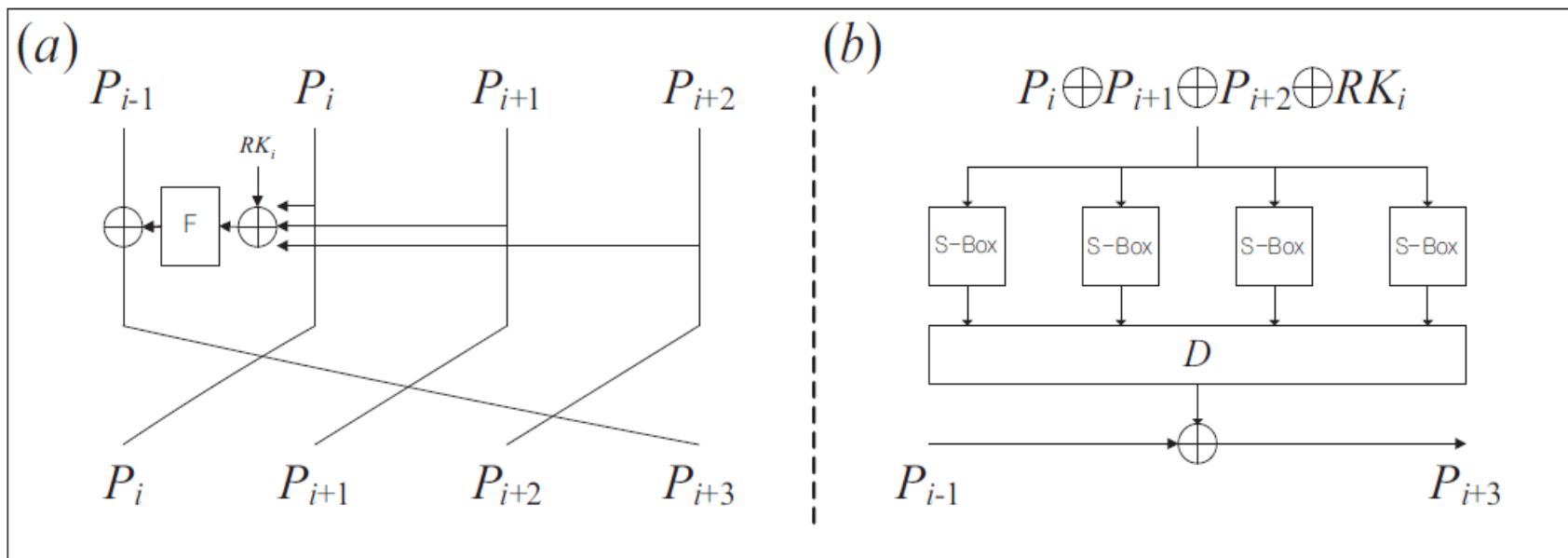


SM4算法

- 我国WAPI无线网络标准中使用的加密算法
- 2012年国密标准，2016年国家标准
- 32轮，迭代的多分枝Feistel结构
- 密钥长度和分组长度都是128比特



SM4算法-轮函数



$$P_{i+3} = P_{i-1} \oplus D(S(P_i \oplus P_{i+1} \oplus P_{i+2} \oplus RK_i))$$

$$D(x) = x \oplus (x \lll 2) \oplus (x \lll 10) \oplus (x \lll 18) \oplus (x \lll 24)$$



SM4算法-S盒

	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xa	0xb	0xc	0xd	0xe	0xf
0x0	d6	90	e9	fe	cc	e1	3d	b7	16	b6	14	c2	28	fb	2c	05
0x1	2b	67	9a	76	2a	be	04	c3	aa	44	13	26	49	86	06	99
0x2	9c	42	50	f4	91	ef	98	7a	33	54	0b	43	ed	cf	ac	62
0x3	e4	b3	1c	a9	c9	08	e8	95	80	df	94	fa	75	8f	3f	a6
0x4	47	07	a7	fc	f3	73	17	ba	83	59	3c	19	e6	85	4f	a8
0x5	68	6b	81	b2	71	64	da	8b	f8	eb	0f	4b	70	56	9d	35
0x6	1e	24	0e	5e	63	58	d1	a2	25	22	7c	3b	01	21	78	87
0x7	d4	00	46	57	9f	d3	27	52	4c	36	02	e7	a0	c4	c8	9e
0x8	ea	bf	8a	de	40	c7	38	b5	a3	f7	f2	ce	f9	61	15	a1
0x9	e0	ae	5d	a4	9b	34	1a	55	ad	93	32	30	f5	8c	b1	e3
0xa	1d	f6	e2	2e	82	66	ca	60	c0	29	23	ab	0d	53	4e	6f
0xb	d5	db	37	45	de	fd	8e	2f	03	ff	6a	72	6d	6c	5b	51
0xc	8d	1b	af	92	bb	dd	bc	7f	11	d9	5c	41	1f	10	5a	d8
0xd	0a	c1	31	88	a5	cd	7b	bd	2d	74	d0	12	b8	e5	b4	b0
0xe	89	69	97	4a	0c	96	77	7e	65	b9	f1	09	c5	6e	c6	84
0xf	18	f0	7d	ec	3a	dc	4d	20	79	ee	5f	3e	d7	cb	39	48

例： $S(0x01)=0x90$



SM4算法-密钥扩展算法

□ 同轮函数相似

□ 128比特的主密钥(MK_0, MK_1, MK_2, MK_3)

$$(K_0, K_1, K_2, K_3) = (MK_0 \oplus T_0, MK_1 \oplus T_1, MK_2 \oplus T_2, MK_3 \oplus T_3,)$$

$$RK_j = K_{j+3} = K_{j-1} \oplus D'(S(K_j \oplus K_{j+1} \oplus K_{j+2} \oplus CK_j))$$

$$D'(x) = x \oplus (x \lll 13) \oplus (x \lll 23)$$

$$CK_j = ((28gj), (28gj + 7), (28gj + 14), (28gj + 21))$$

$$T_0 = 0xa3b1bac6, T_1 = 0x56aa3350, T_2 = 0x677d9197, T_3 = 0xb27022dc.$$



AES History

□ AES

- 1997年，NIST公开征集数据加密算法以取代DES
- 1998年，共收到15个算法
- 1999年，从15个中选中5个算法：MARS、RC6、Rijndael、Serpent和Twofish
- 2000年10月：Rijndael获胜，Vincent Rijmen和Joan Daem
- 2001年11月：AES



的数据加密标准



高级数据加密标准AES

□ AES

- 分组密码
- 分组长度128比特
- Substitution-Permutation Network (SPN)
- 三种不同长度的密钥和轮数
 - AES-128: 128比特密钥 + 10轮
 - AES-192: 192比特密钥 + 12轮
 - AES-256: 256比特密钥 + 14轮



AES 应用

□ AES

- 免费使用
- 简单漂亮的设计
- 安全性高
- 实现效率高

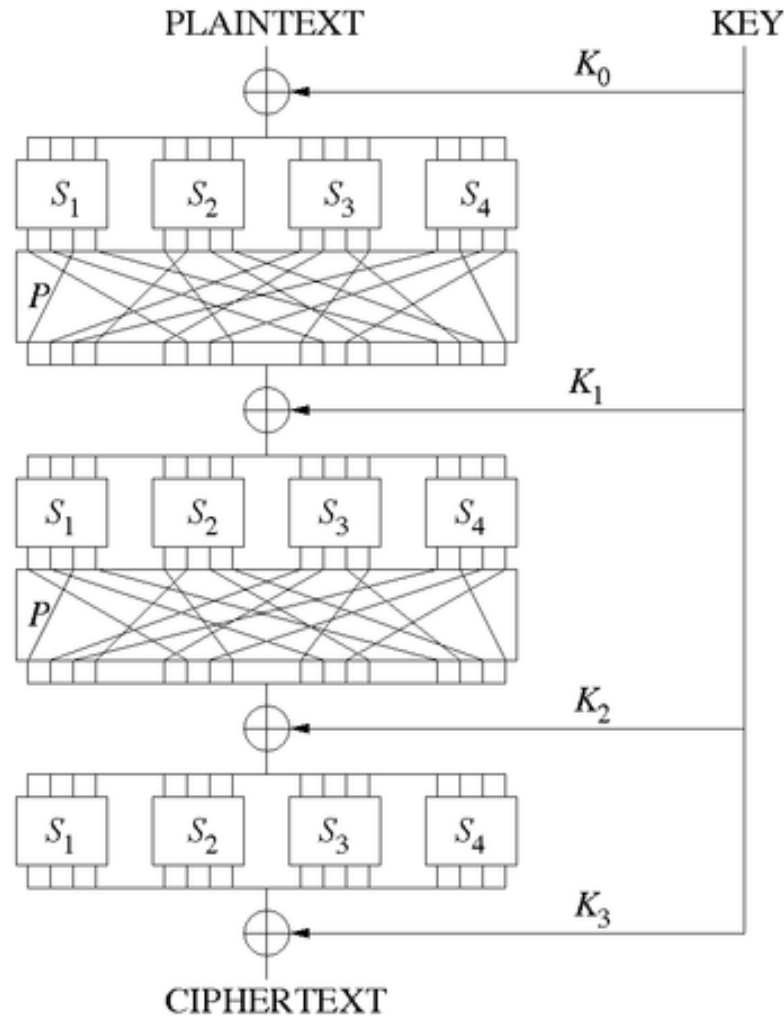
□ 美国国家标准

- AES-128用于SECRET信息
- AES-192和AES-256用于TOP SECRET 信息

□ 商业应用

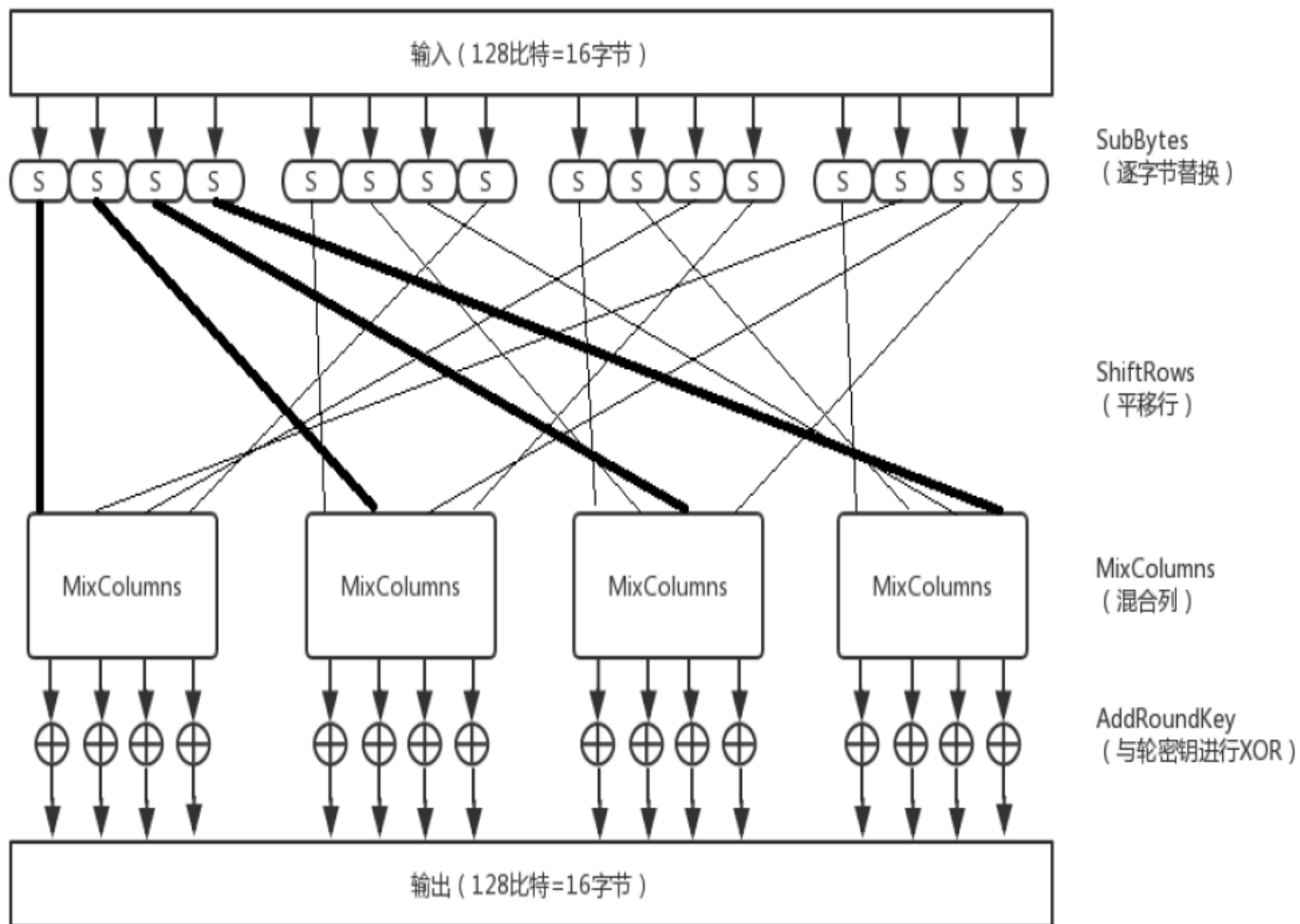


Substitution-Permutation Network (SPN)



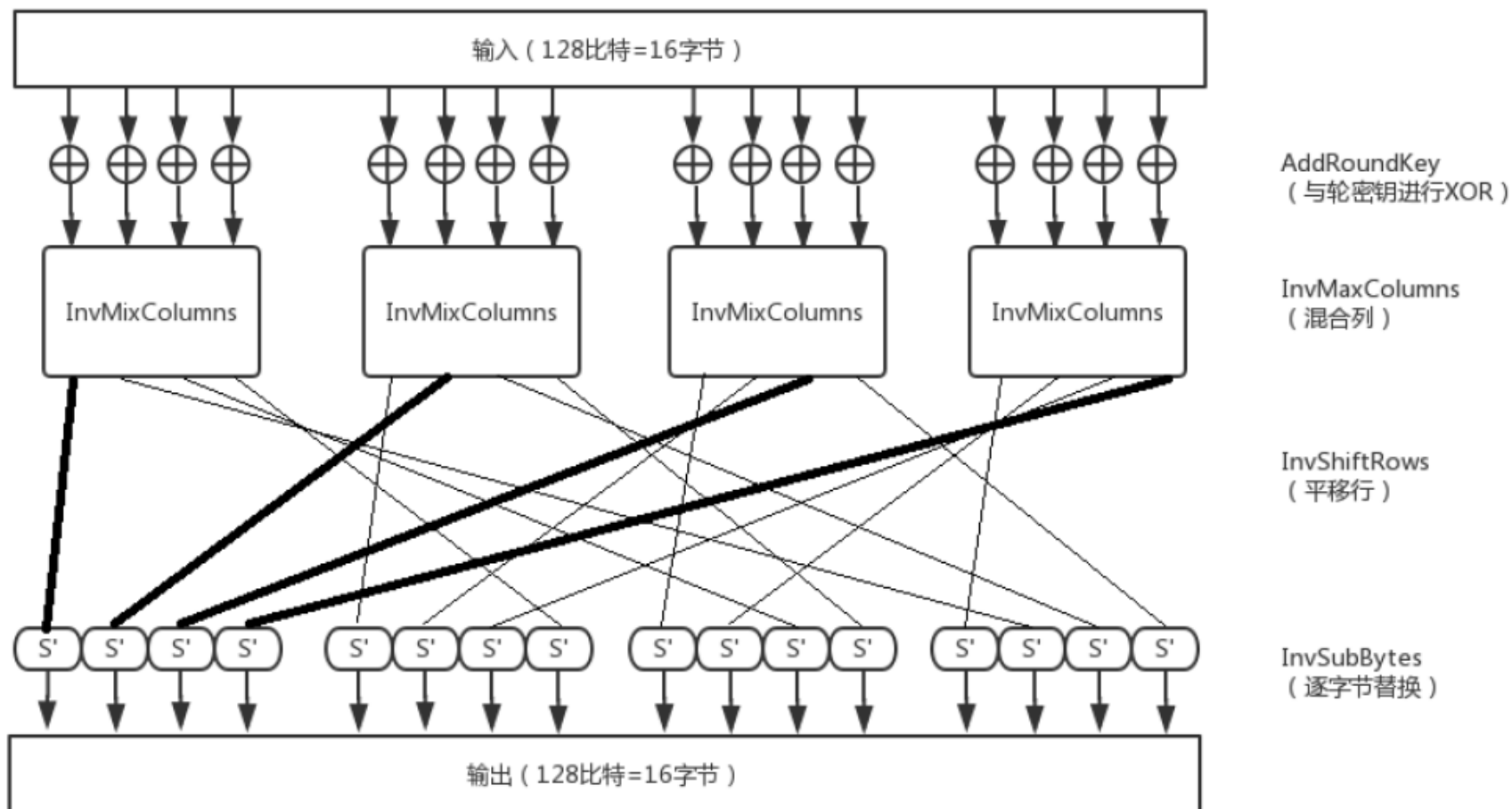


Rijndael加密中的一轮





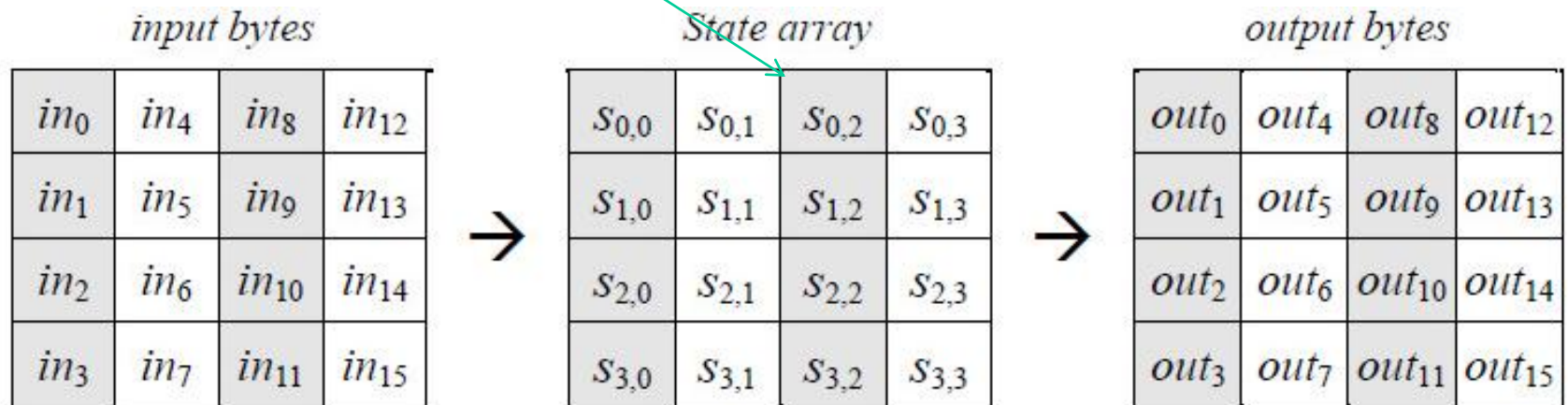
Rijndael解密中的一轮





AES算法：State

- 状态(State): 与消息分组相同，16个字节
- 表示成4乘4的矩阵





AES算法：总体

State=Plaintext

AddRoundKey(State, Key₀)

For i=0 to r-1

SubBytes(State)

ShiftRows(State)

MixColumns(State)

AddRoundKey(State, RoundKey_i)

End for

r-1轮

$s_{0,0}$	$s_{0,1}$	$s_{0,2}$	$s_{0,3}$
$s_{1,0}$	$s_{1,1}$	$s_{1,2}$	$s_{1,3}$
$s_{2,0}$	$s_{2,1}$	$s_{2,2}$	$s_{2,3}$
$s_{3,0}$	$s_{3,1}$	$s_{3,2}$	$s_{3,3}$

SubBytes(State)

~~ShiftRows(State)~~

MixColumns(State)

AddRoundKey(State, RoundKey_r)

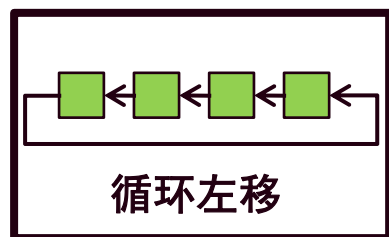
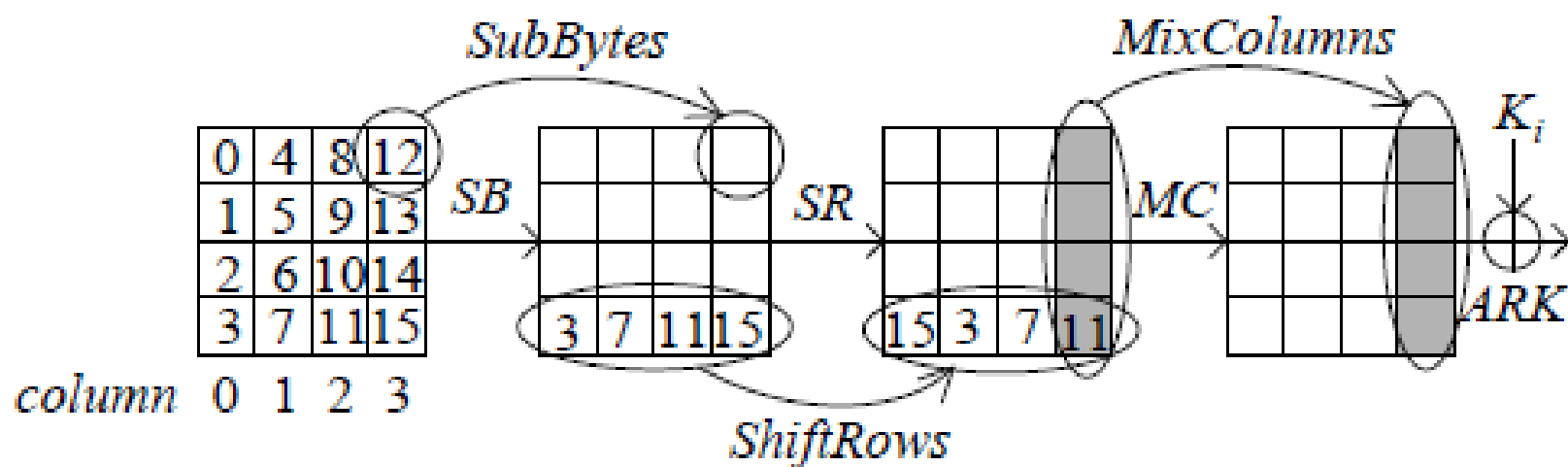
最后轮

Ciphertext=State

需要 $r+1$ 个轮密钥



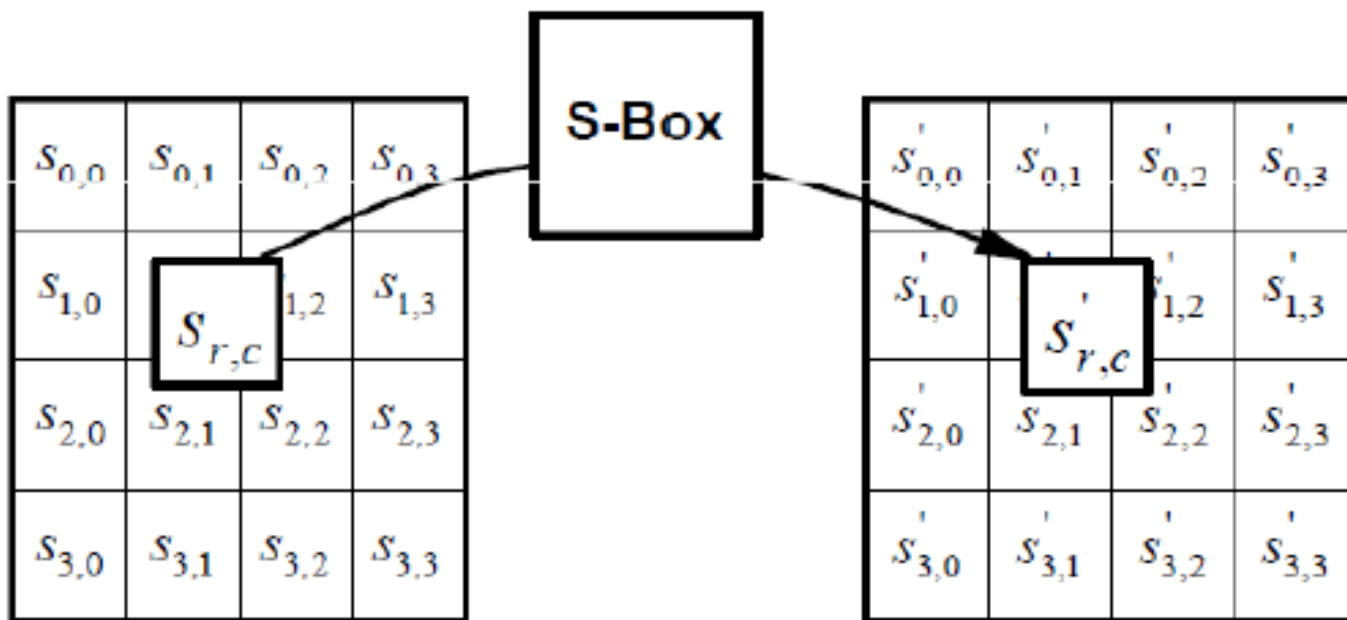
AES算法一轮





AES: SubBytes

- 字节代替变换(SubBytes()): 对每个字节进行S盒查表代换





AES: SubBytes(续)

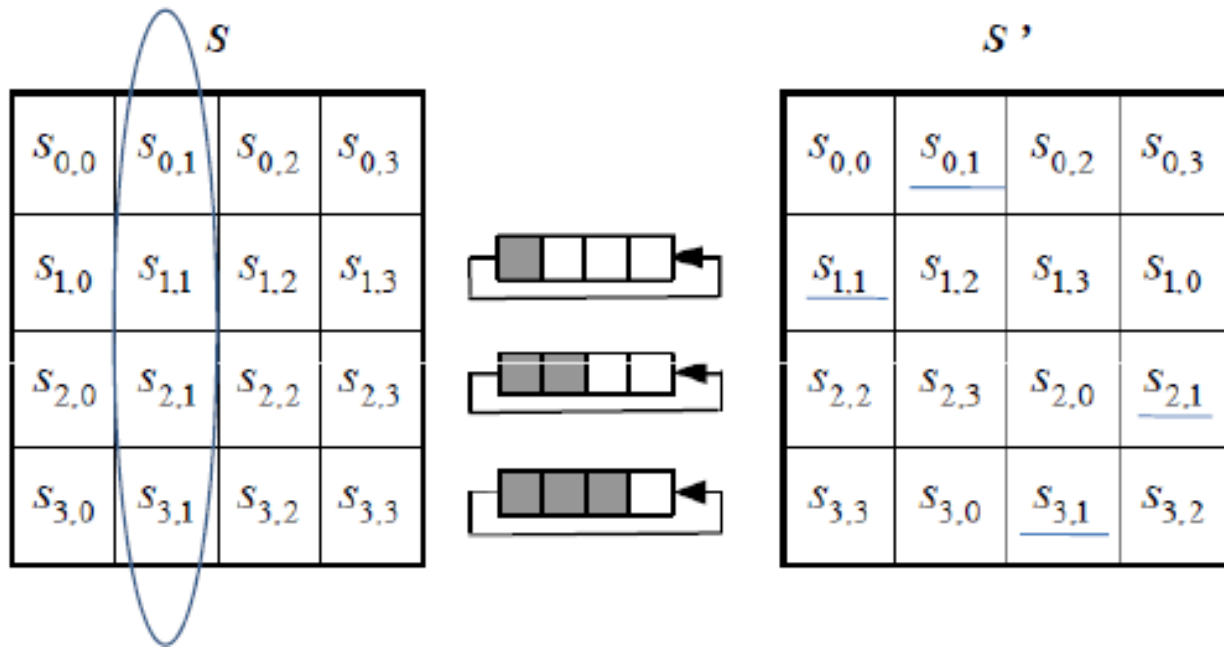
- S-box: 8比特输入、8比特输出、可逆
- 由以下两个步骤计算 $b' = S(a)$
 - 在 $GF(2^8)$ 中求 $b = a^{-1}$ (使用扩展Euclid算法)
 - 对 b 应用以下仿射变换

$$\begin{bmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \\ b'_4 \\ b'_5 \\ b'_6 \\ b'_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$



AES: ShiftRows

行移位变换(ShiftRows)



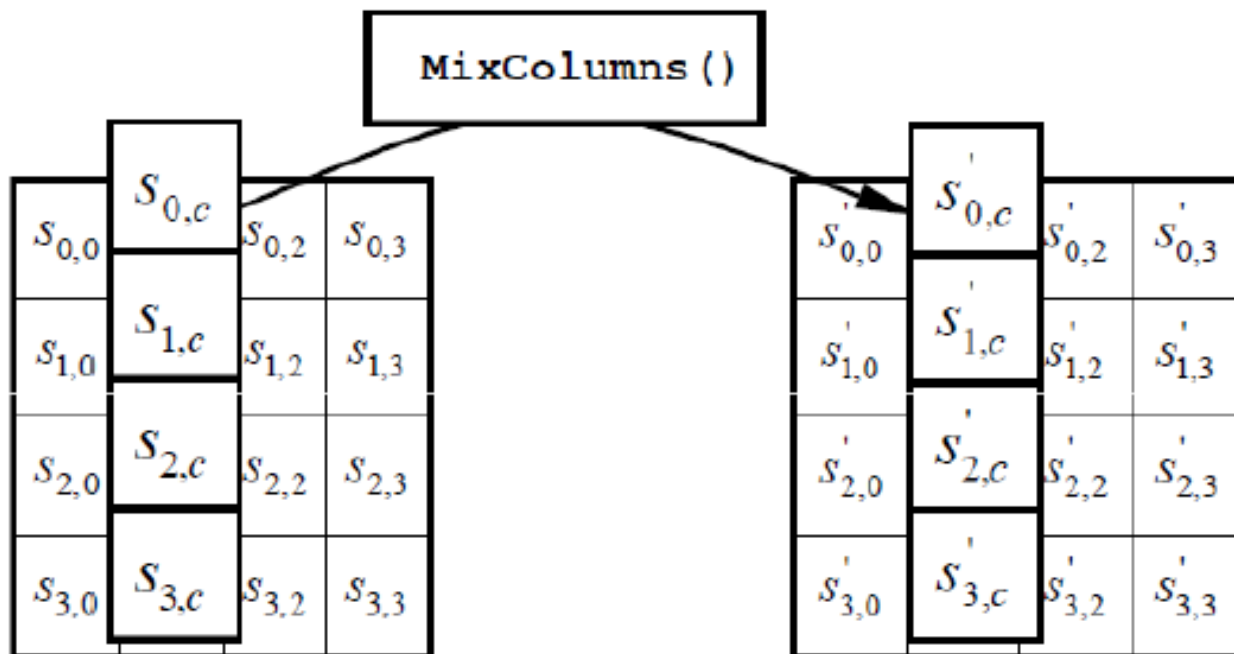
第1行:不移位
第2行:左移1位
第3行:左移2位
第4行:左移3位

经过行移位后，1列中的4个字节被分布到不同的列中



AES: MixColumns

□ 列混合变换(MixColumns()): 对一个状态逐列进行变换



$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$$

$$s'(x) = a(x) \otimes s(x)$$



AES: MixColumns

列混合

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} = \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$$



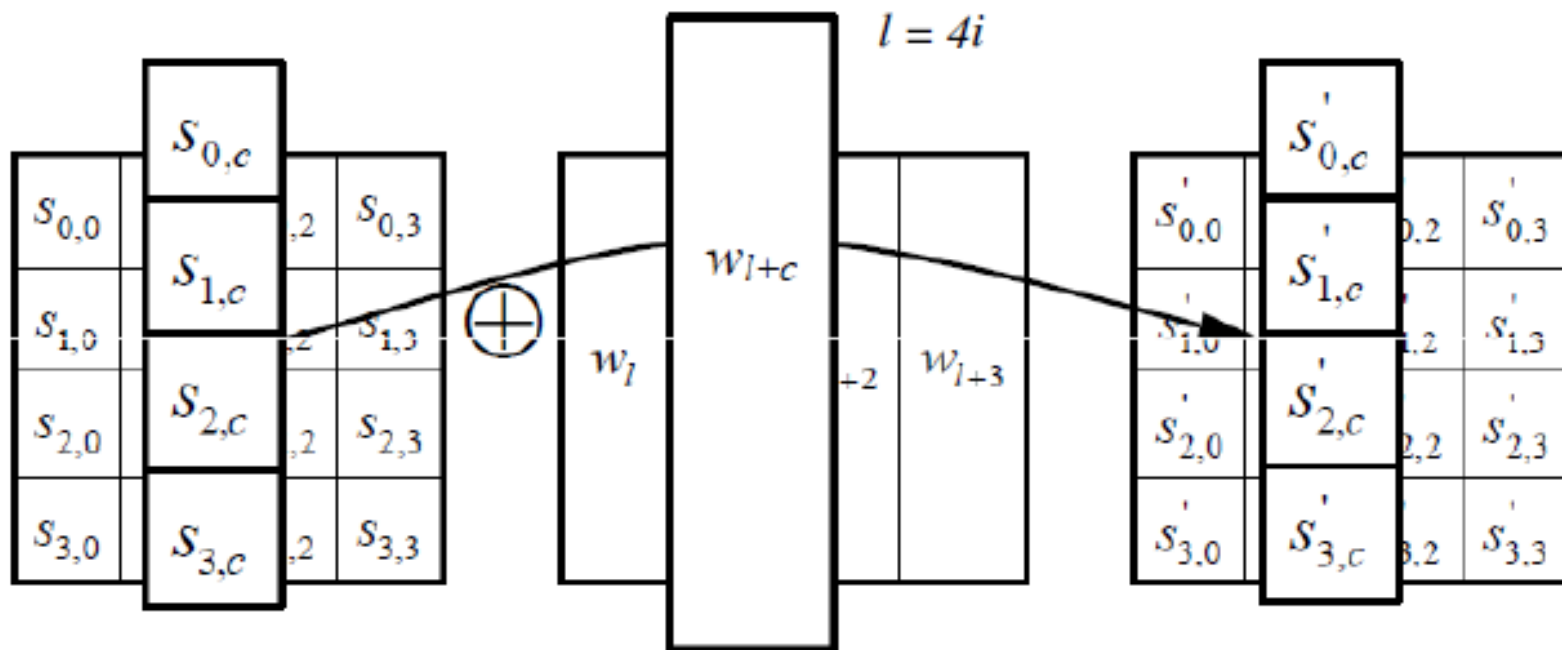
AES: MixColumns

□ 列混和的一些属性

- 一个输入的字节影响所有的4个输出字节
- 假设有 t_1 个非零输入字节，输出有 t_2 个非零字节，则 $t_1 + t_2 \geq 5$
- 最大距离可分码(MDS): 对任意的 x , 则在 $GF(2^8)$ 中, $(x, \text{MixColumns}(x))$ 至少是5。



AES: AddRoundKey





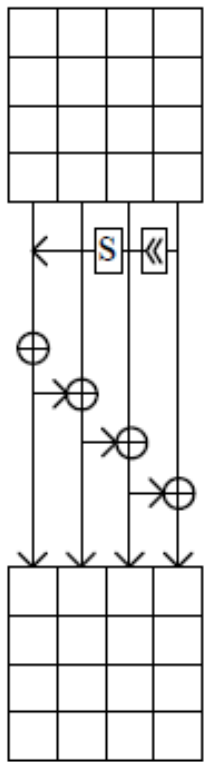
AES:密钥生成方案

- 每个轮密钥128比特
- 轮密钥用32比特字的数组来表示 $w[i]$
 - 第一轮: $w[0], w[1], w[2], w[3]$
 - 第二轮: $w[4], w[5], w[6], w[7]$
 -

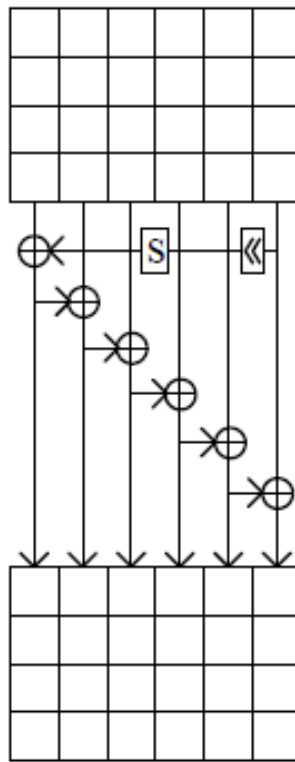


AES算法密钥生成

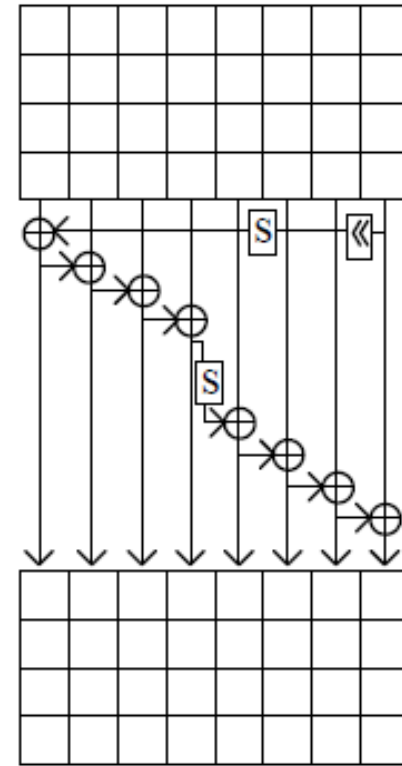
□ 密钥调度



AES-128



AES-192



AES-256



AES: 密钥生成方案

- Example: AES-128

```
RCon[1] ← 01000000  
RCon[2] ← 02000000  
RCon[3] ← 04000000  
RCon[4] ← 08000000  
RCon[5] ← 10000000  
RCon[6] ← 20000000  
RCon[7] ← 40000000  
RCon[8] ← 80000000  
RCon[9] ← 1B000000  
RCon[10] ← 36000000
```

Round constants

```
W[0] = (K[0],K[1],K[2],K[3])  
W[1] = (K[4],K[5],K[6],K[7])  
W[2] = (K[8],K[9],K[10],K[11])  
W[3] = (K[12],K[13],K[14],K[15])
```

```
for i ← 0 to 3
```

```
do w[i] ← (key[4i], key[4i + 1], key[4i + 2], key[4i + 3])
```

Load the key into w[]

```
for i ← 4 to 43
```

```
do { temp ← w[i - 1]  
    if i ≡ 0 (mod 4)  
    then temp ← SUBWORD(ROTWORD(temp)) ⊕ RCon[i/4]  
    w[i] ← w[i - 4] ⊕ temp
```

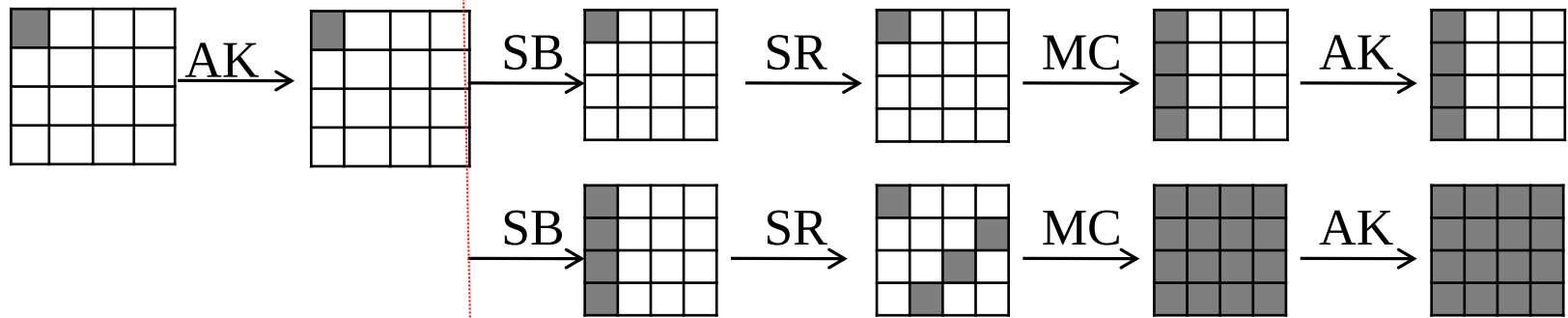
S盒变换

循环左移1个字节

```
return (w[0],w[1],...,w[43])
```



□ AES两轮





AES加解密

State=Plaintext

AddRoundKey(State, Key₀)

For i=0 to r-1

Subbytes(State)

ShiftRows(State)

MixColumns(State)

AddRoundKey(State, RoundKey_i)

End for

Subbytes(State)

ShiftRows(State)

~~MixColumns(State)~~

AddRoundKey(State, RoundKey_r)

Ciphertext=State

State=Ciphertext

AddRoundKey(State, RoundKey_r)

For i=0 to r-1

InvShiftRows(State)

InvSubBytes(State)

AddRoundKey(State, RoundKey_{r-i})

InvMixColumns(State)

End for

InvShiftRows(State)

InvSubBytes(State)

AddRoundKey(State, RoundKey_{r-i})

~~InvMixColumns(State)~~

Plaintext=State



AES数学基础

Euclid算法(辗转相除法)：设 a 和 b 是给定的两个整数， $b \neq 0$, b 不能整除 a , 重复应用带余除法得到下列 k 个等式：

$$a = q_0 b + r_0, \quad 0 < r_0 < |b|$$

$$b = q_1 r_0 + r_1, \quad 0 < r_1 < r_0$$

$$r_0 = q_2 r_1 + r_2, \quad 0 < r_2 < r_1,$$

.....

$$r_{k-5} = q_{k-3} r_{k-4} + r_{k-3}, \quad 0 < r_{k-3} < r_{k-4}$$

$$r_{k-4} = q_{k-2} r_{k-3} + r_{k-2}, \quad 0 < r_{k-2} < r_{k-3}$$

$$r_{k-3} = q_{k-1} r_{k-2}$$

则 $r_{k-2} = \text{GCD}(a, b)$ 。复杂度 $O(\log_2^2 a)$



AES数学基础（续）

1. 欧几里德算法(Euclid)，辗转相除法

□ 用于计算 a 和 b 的最大公因子 $\text{GCD}(a,b)$

□ 使用 $\text{GCD}(a,b) = \text{GCD}(b, a \bmod b)$

$$a = q_0 b + r_0, \quad 0 < r_0 < |b|$$

$$b = q_1 r_0 + r_1, \quad 0 < r_1 < r_0$$

$$r_0 = q_2 r_1 + r_2, \quad 0 < r_2 < r_1$$

.....

$$r_{k-3} = q_{k-1} r_{k-2} + r_{k-1}, \quad 0 < r_{k-1} < r_{k-2}$$

$$r_{k-2} = q_k r_{k-1} + r_k, \quad 0 < r_k < r_{k-1}$$

$$r_{k-1} = q_{k+1} r_k$$

2. 扩展的欧基里德算法

□ 用于寻找 x 和 y , 满足 $ax + by = \text{GCD}(a,b)$

□ 基本思想：在欧几里德算法的第 i 步，寻找 $r_i = ax_i + by_i$.



AES数学基础（续）

□ 例：求42823和6409的最大公因子，并将它表示成42823和6409的整系数线形组合形式。

$$42823 = 6 \cdot 6409 + 4369$$

$$6409 = 1 \cdot 4369 + 2040$$

$$4369 = 2 \cdot 2040 + 289$$

$$2040 = 7 \cdot 289 + 17$$

$$289 = 7 \cdot 17$$

$$(42823, 6409)$$

$$=(6409, 4369)$$

$$=(4369, 2040)$$

$$=(2040, 289)$$

$$=(289, 17) = 17$$

• 上面过程的逆过程

$$17 = 2040 - 7 \cdot 289$$

$$\begin{aligned} 17 &= 2040 - 7 \cdot (4369 - 2 \cdot 2040) \\ &= -7 \cdot 4369 + 15 \cdot 2040 \end{aligned}$$

$$\begin{aligned} 17 &= -7 \cdot 4369 + 15 \cdot (6409 - 4369) \\ &= 15 \cdot 6409 - 22 \cdot 4369 \end{aligned}$$

$$\begin{aligned} 17 &= 15 \cdot 6409 - 22 \cdot (42823 - 6 \cdot 6409) \\ &= -22 \cdot 42823 + 147 \cdot 6409 \end{aligned}$$

$$\text{即}(42823, 6409)$$

$$= -22 \cdot 42823 + 147 \cdot 6409.$$



AES数学基础（续）

□群： 设 G 是一个非空集合，在 G 中定义了一个二元运算 $\circ$ ，若 $\circ$ 满足下面条件，则 G 称为一个群。

1. 任意 $a, b \in G$, 则 $a \circ b \in G$ } 半群

2. 对于任意的 $a, b, c \in G$, $(a \circ b) \circ c = a \circ (b \circ c)$

3. 在 G 中存在一个元素 e ，它对 G 中任何一个元素 g ，有

$$e \circ g = g \circ e = g$$

4. 对 G 中任何一个元素 g ，都存在一个元素 g' ，使得

$$g \circ g' = g' \circ g = e$$

e 唯一，称为单位元

g' 唯一，称为 g 的逆元



AES数学基础（续）

□例：

如整数集合 Z 对 数的加法 构成一个群；

全体不等于0的有理数对普通数的乘法构成一个群

设 $n \in Z$ ， 模 n 剩余类 $Z_n = \{[k] | k \in Z\} = \{\overline{0}, \overline{1}, \dots, \overline{n-1}\}$ 对模 n 加法构成一个群

交换群：

一个群，如果对所有的 $a \circ b \in G$, 都有 $a \circ b = b \circ a$, 则称 G 是一个交换群（Abel）如加群



AES数学基础（续）

□环

设 R 是一个非空集合，在其上定义两种运算加法 $(+)$ 和乘法 $(\bullet)$ ，如果这些运算满足

1. $(R, +)$ 是一个加群，即 $(R, +)$ 对叫加法做成一个交换群
2. $(R, \bullet)$ 对另一个叫乘法的运算做成一个半群
3. 加法对乘法的左右分配律成立：对任意的 $a, b, c \in R$,

$$a \bullet (b + c) = a \bullet b + a \bullet c$$

$$(b + c) \bullet a = b \bullet a + c \bullet a$$

则称 $(R, +, \bullet)$ 是一个环。记为 R



AES数学基础（续）

□环 例子：

1. 整数 $\mathbb{Z}$ 对数的加法和乘法做成一个环，称为整数环
2. 模 n 剩余类对模加法和模 n 乘法成为一个环。

交换环： 一个环 R 成为一个交换环，若

$$\forall a, b \in R, a \bullet b = b \bullet a$$

单位元： 一个环 R 的一个元素 e 叫做一个单位元，若

$$\forall a \in R, e \bullet a = a \bullet e = a$$

逆元： 一个有单位元环的一个元 b 称为 a 的一个逆元，假如

$$a \bullet b = b \bullet a = e$$



AES数学基础（续）

定义： 一个至少含有两个元素的环 R 叫做域，假如

1. R 是交换环。
2. R 有一个单位元。
3. R 的每一个不等于0的元有一个逆元。

则 R 为域，记 R 为 F

等价定义： 一个至少含有两个元素的集合 F 定义了两种运算 $+$ 和 $*$ ，如果

1. $(F, +)$ 为一个可换加群。
2. $(F^*, \bullet)$ 为一个可换乘群， F^* 表示 F 中所有的非零元，则 F 为域。
3. 加法对乘法满足分配律。

如全体有理数的集合、全体实数、全体复数按普通意义下的加、乘构成域：有理数域、实数域和复数域



AES数学基础（续）

- 有限域(Galois Field)
- 一个只含有有限个元素的域
- 一个阶为 m 的有限域存在，当且仅当存在一个素数 p 和一个正整数 n , 使得 $m=p^n$
- 例如： $GF(7)=\{0,1,2,3,4,5,6\}$
 - 模7整数加
 - 模7整数乘
- $GF(p^n)$: 系数在 p 上的次数为 $n-1$ 的多项式的集合



Évariste Galois
(1811-1832)



AES数学基础（续）

□有限域 $GF(2^8)$ ：一个 $GF(2^8)$ 中的元素的两种表示方式

□二进制表示： $b = b_7b_6b_5b_4b_3b_2b_1b_0$

□多项式表示

$$b_7x^7 + b_6x^6 + b_5x^5 + b_4x^4 + b_3x^3 + b_2x^2 + b_1x + b_0$$

□例如16进制数0x57的二进制表示01010111，对应的多项式为

$$x^6 + x^4 + x^2 + x + 1$$



AES数学基础（续）

□ 有限域 $GF(2^8)$ 中两个元素的加

$$(x^6 + x^4 + x^2 + x + 1) \oplus (x^7 + x + 1) = x^7 + x^6 + x^4 + x^2$$

$$\{01010111\} \oplus \{10000011\} = \{11010100\}$$

$$\{57\} \oplus \{83\} = \{64\}$$

• 有限域 $GF(2^8)$ 中两个元素的乘法

- 用●表示
- 模二元域 $GF(2)$ 上一个8次不可约多项式的乘积
- AES选择不可约多项式为

$$m(x) = x^8 + x^4 + x^3 + x + 1$$



AES数学基础（续）

□有限域GF(2⁸)上两个元素的乘法，例

$$\begin{aligned} (x^6 + x^4 + x^2 + x + 1) \bullet (x^7 + x + 1) &= x^{13} + x^{11} + x^9 + x^8 + x^7 + \\ &\quad x^7 + x^5 + x^3 + x^2 + x + \\ &\quad x^6 + x^4 + x^2 + x + 1 \\ &= x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1 \\ &= x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1 \bmod (x^8 + x^4 + x^3 + x + 1) \\ &= x^7 + x^6 + 1 \\ \{57\} \bullet \{83\} &= \{c1\} \end{aligned}$$



AES数学基础（续）

□ x 与多项式 $b(x)$ 的乘积

$$b_7x^8 + b_6x^7 + b_5x^6 + b_4x^5 + b_3x^4 + b_2x^3 + b_1x^2 + b_0x$$

□ 计算 $x \bullet b(x) \bmod m(x)$

$$\text{xtime}(b) = '02' \bullet b = \begin{cases} b \ll 1, & \text{if } b_7 = 0 \\ (b \ll 1) \oplus 1B, & \text{if } b_7 = 1 \end{cases}$$

□ 例： $'57' \bullet '13' = 'FE'$

$$'13' = '01' \oplus '02' \oplus '10'$$

$$'57' \bullet '02' = \text{xtime}('57') = 'AE'$$

$$'57' \bullet '04' = \text{xtime}('AE') = '47'$$

$$'57' \bullet '08' = \text{xtime}('47') = '8E'$$

$$'57' \bullet '10' = \text{xtime}('8E') = '07'$$

$$'57' \bullet '13' = '57' \bullet ('01' \oplus '02' \oplus '10')$$

$$= '57' \oplus 'AE' \oplus '07'$$

$$= 'FE'$$



AES数学基础（续）

□ 有限域 $GF(2^8)$ 中乘法逆元的求解：使用扩展欧几里德算法

□ 给定 a 和 b , 寻找 x 和 y , 满足

$$ax + by = \gcd(a, b)$$

□ 若 $\gcd(a, b) = 1$, 则 $ax \bmod b = 1$

即 x 是 $a \bmod b$ 的乘法逆元



AES数学基础（续）

□系数在 $GF(2^8)$ 中的多项式

□设 $[a_0, a_1, a_2, a_2]$ 是4个字节(bytes), 对应着一个系数在 $GF(2^8)$, 次数小于4的多项式

$$a(x) = a_3x^3 + a_2x^2 + a_1x + a_0$$

□上面的多项式与 $GF(2^8)$ 中的多项式是不同的

□系数在 $GF(2^8)$

□使用一个不同的模多项式: $M(x) = x^4 + 1$

□ x^4+1 不是 $GF(2^8)$ 上的一个不可约多项式

□一个固定多项式模 $M(x)$ 不一定有乘法逆元

□AES中选择了有一个乘法逆元的固定多项式



AES数学基础（续）

□ 系数在 $GF(2^8)$ 中的多项式的加法运算

$$a(x) = a_3x^3 + a_2x^2 + a_1x + a_0$$

$$b(x) = b_3x^3 + b_2x^2 + b_1x + b_0$$

$$a(x) + b(x) = (a_3 \oplus b_3)x^3 + (a_2 \oplus b_2)x^2 + (a_1 \oplus b_1)x + (a_0 \oplus b_0)$$



AES数学基础（续）

□ 系数在 $GF(2^8)$ 中的多项式的乘法运算： $\otimes$

$$a(x) = a_3x^3 + a_2x^2 + a_1x + a_0$$

$$b(x) = b_3x^3 + b_2x^2 + b_1x + b_0$$

$$c(x) = a(x) \bullet b(x)$$

$$= c_6x^6 + c_5x^5 + c_4x^4 + c_3x^3 + c_2x^2 + c_1x + c_0$$

$$c_0 = a_0 \bullet b_0$$

$$c_1 = a_1 \bullet b_0 \oplus a_0 \bullet b_1$$

$$c_2 = a_2 \bullet b_0 \oplus a_1 \bullet b_1 \oplus a_0 \bullet b_2$$

$$c_3 = a_3 \bullet b_0 \oplus a_2 \bullet b_1 \oplus a_1 \bullet b_2 \oplus a_0 \bullet b_3$$

$$c_4 = a_3 \bullet b_1 \oplus a_2 \bullet b_2 \oplus a_1 \bullet b_3$$

$$c_5 = a_3 \bullet b_2 \oplus a_2 \bullet b_3$$

$$c_6 = a_3 \bullet b_3$$



AES数学基础（续）

□系数在 $GF(2^8)$ 中的多项式的乘法运算（续）

$$\begin{aligned}d(x) &= a(x) \otimes b(x) \\ &= a(x) \bullet b(x) \bmod (x^4 + 1)\end{aligned}$$

$$\text{由于 } x^i \bmod (x^4 + 1) = x^{i \bmod 4}$$

$$\text{故 } d(x) = c_3 x^3 + (c_6 \oplus c_2) x^2 + (c_5 \oplus c_1) x + (c_4 \oplus c_0)$$



AES数学基础（续）

□系数在 $GF(2^8)$ 中的多项式的乘法运算（续）

设

$$d(x) = d_3x^3 + d_2x^2 + d_1x + d_0$$

则

$$d_0 = a_0 \bullet b_0 \oplus a_3 \bullet b_1 \oplus a_2 \bullet b_2 \oplus a_1 \bullet b_3$$

$$d_1 = a_1 \bullet b_0 \oplus a_0 \bullet b_1 \oplus a_3 \bullet b_2 \oplus a_2 \bullet b_3$$

$$d_2 = a_2 \bullet b_0 \oplus a_1 \bullet b_1 \oplus a_0 \bullet b_2 \oplus a_3 \bullet b_3$$

$$d_3 = a_3 \bullet b_0 \oplus a_2 \bullet b_1 \oplus a_1 \bullet b_2 \oplus a_0 \bullet b_3$$



AES数学基础（续）

□系数在 $GF(2^8)$ 中的多项式的乘法运算（续）

对一个固定的多项式

$$a(x), d(x) = a(x) \otimes b(x)$$

能够表示成下列矩阵形式

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$



AES数学基础（续）

□ 系数在 $GF(2^8)$ 中的多项式的乘法运算（续）

由于 x^4+1 不是 $GF(2^8)$ 中的不可约多项式，一个固定的多项式模 $M(x)$ 不一定有乘法逆元，AES选择了一个有逆元的固定多项式

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$$

$$a^{-1}(x) = \{0b\}x^3 + \{0d\}x^2 + \{09\}x + \{0e\}$$

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$



谢谢！